

Instituto Politécnico Nacional
Escuela Superior de Cómputo



Ingeniería en Sistemas Computacionales · Plan 2020

Unidad de aprendizaje: **Teoría de la Computación**

Práctica 4

Uso de las Expresiones Regulares y Extensión de Software para Convertir Expresiones Regulares

Alumno: Rico Espluga Juan Diego

Boleta: 2025630474

Grupo: 4CV4

Docente: Gabriel Hurtado Avilés

Fecha de entrega: 6 de noviembre de 2026

Índice

1. Objetivo **3**

2. Ejercicio 1. Investigación sobre expresiones regulares **3**

2.1. Lenguajes regulares y expresiones regulares 3

2.2. Lugar en la jerarquía de Chomsky 3

2.3. Teorema de Kleene 4

2.4. Leyes algebraicas usadas en la Práctica 3 4

2.5. Expresiones de la teoría y de los lenguajes de programación 5

2.6. Cinco aplicaciones 5

3. Ejercicio 2. De expresión regular a autómata **6**

3.1. Notación aceptada 6

3.2. Análisis: *shunting-yard* con posiciones 7

3.3. Parte I del teorema de Kleene, paso a paso 8

3.4. Validación de palabras 9

3.5. Casos de prueba 10

4. Ejercicio 3. De autómata a expresión regular **12**

4.1. Verificación del determinismo 12

4.2. Eliminación de estados paso a paso 12

4.3. Comparación con JFLAP 12

4.4. Casos de prueba 14

5. Ejercicio 4. Aplicaciones: validadores **16**

5.1. Correo electrónico 16

5.2. Teléfono en formato nacional (México) 17

5.3. URL (http y https) 18

5.4. Tamaño de los autómatas 18

5.5. Congruencia entre modelos 19

6. Conclusiones **20**

1. Objetivo

Estudiar las expresiones regulares y sus aplicaciones, e incorporar al software las dos direcciones del teorema de Kleene: de una expresión regular a un autómata y de un autómata a una expresión regular.

La práctica tiene cuatro ejercicios y reutiliza la eliminación de λ, la construcción de subconjuntos y la minimización de la Práctica 3. Los casos de prueba están en `automatas/practica4/` y el reporte en Markdown en `docs/practica4/`.

2. Ejercicio 1. Investigación sobre expresiones regulares

2.1. Lenguajes regulares y expresiones regulares

Sobre un alfabeto Σ, las **expresiones regulares** se definen por inducción (Hopcroft et al., 2007, sec. 3.1.2): $\varnothing$, λ y cada símbolo $a \in \Sigma$ son expresiones, con $L(\varnothing) = \varnothing$, $L(\lambda) = \{\lambda\}$ y $L(a) = \{a\}$; y si r y s son expresiones, también lo son $r + s$, rs, r^* y (r), con

- **unión**: $L(r + s) = L(r) \cup L(s)$;
- **concatenación**: $L(rs) = L(r)L(s) = \{xy : x \in L(r), y \in L(s)\}$;
- **cerradura de Kleene**: $L(r^*) = \bigcup_{n \geq 0} L(r)^n$, con $L(r)^0 = \{\lambda\}$.

Un lenguaje es **regular** si lo describe alguna expresión regular; equivalentemente, si lo reconoce algún autómata finito. La distinción importa: la expresión es un objeto sintáctico (una cadena de símbolos con operadores) y el lenguaje es un conjunto, posiblemente infinito. Muchas expresiones distintas describen el mismo lenguaje, como $(a + b)^*$ y $(a^*b^*)^*$, y decidir si dos lo hacen es justamente lo que permiten los autómatas mínimos. La precedencia es estrella, después concatenación y al final unión, de modo que $ab^* + c$ significa $(a(b^*)) + c$.

2.2. Lugar en la jerarquía de Chomsky

Los lenguajes regulares son el **tipo 3**, el nivel más bajo de la jerarquía (Chomsky, 1956): los generan las gramáticas lineales por la derecha (producciones $A \to aB$ o $A \to a$, y $S \to \lambda$) y los reconocen los autómatas finitos. Están contenidos estrictamente en los independientes del contexto

(tipo 2): $\{a^n b^n : n \geq 0\}$ lo genera $S \to aSb \mid \lambda$, pero ningún autómata finito lo reconoce. La razón es la memoria: un autómata finito sólo distingue un número fijo de situaciones, y contar n aes no acotadas exige una pila.

2.3. Teorema de Kleene

Teorema (Kleene, 1956; Hopcroft et al., 2007, sec. 3.2, teoremas 3.4 y 3.7; Sipser, 2013, sec. 1.3, teorema 1.54). Un lenguaje es regular si y sólo si es reconocido por un autómata finito.

- **De expresión a autómata** (Parte I en la notación del curso): por inducción sobre la expresión se construye un AFN-λ con un único estado inicial y un único final; hay un caso básico para $\varnothing$, λ y cada símbolo, y una construcción para cada operador. El Ejercicio 2 la implementa.
- **De autómata a expresión** (Parte II): se obtiene la expresión de un AFD con las expresiones $R_{ij}^{(k)}$, eliminando estados o resolviendo el sistema de ecuaciones con el lema de Arden. El Ejercicio 3 implementa la eliminación de estados y el lema de Arden.

2.4. Leyes algebraicas usadas en la Práctica 3

Para simplificar las expresiones del Ejercicio 1 de la Práctica 3 se usaron estas leyes (Hopcroft et al., 2007, sec. 3.4):

Tabla 1. Leyes algebraicas.

Ley	Forma
Conmutatividad y asociatividad de la unión	$r + s = s + r$; $(r + s) + t = r + (s + t)$
Asociatividad de la concatenación	$(rs)t = r(st)$
Idempotencia	$r + r = r$
Neutros y absorbente	$\varnothing + r = r$; $\lambda r = r\lambda = r$; $\varnothing r = r\varnothing = \varnothing$
Distributividad	$r(s + t) = rs + rt$; $(s + t)r = sr + tr$
Estrella	$(r^*)^* = r^*$; $\varnothing^* = \lambda^* = \lambda$; $\lambda + rr^* = r^*$
Desplazamiento	$r(sr)^* = (rs)^*r$
Unión bajo estrella	$(r + s)^* = r^*(sr^*)^*$

2.5. Expresiones de la teoría y de los lenguajes de programación

Las expresiones de los lenguajes de programación (PCRE, el módulo `re` de Python, JavaScript) parten de las de la teoría pero añaden sintaxis cómoda que no cambia la potencia —clases `[0-9]`, cuantificadores `+`, `?`, `{m,n}`, anclas `^` y `$`— y operadores que **sí** la cambian, como las referencias hacia atrás `\1`, que obligan a repetir exactamente el texto que capturó un grupo y con las que se describen lenguajes que no son regulares (Câmpeanu et al., 2003). Además, su significado práctico no es «¿pertenece la cadena al lenguaje?», sino «¿dónde está la primera coincidencia?», y muchos motores la buscan con retroceso, lo que puede tardar un tiempo exponencial (Davis et al., 2018).

El patrón (a*)b\1 no es regular. El grupo captura una sucesión de aes, después debe venir una b y al final la *misma* sucesión. Con coincidencia completa describe $L = \{a^n b a^n : n \geq 0\}$.

Demostración con el lema de bombeo (Hopcroft et al., 2007, sec. 4.1). Supongamos que L es regular y sea p su constante de bombeo. Tomamos $\omega = a^p b a^p \in L$, con $|\omega| \geq p$. El lema garantiza una descomposición $\omega = xyz$ con $|xy| \leq p$, $|y| \geq 1$ y $xy^i z \in L$ para todo $i \geq 0$. Como $|xy| \leq p$, x y y están dentro del primer bloque de aes: $y = a^k$ con $k \geq 1$. Con $i = 2$, $xy^2z = a^{p+k} b a^p$, que tiene más aes antes de la b que después, así que no pertenece a L. Esto contradice el lema; por lo tanto L no es regular y ningún autómata finito, ni ninguna expresión regular de la teoría, lo describe.

2.6. Cinco aplicaciones

Tabla 2. Aplicaciones de las expresiones regulares.

Aplicación	Expresión concreta	Cómo funciona
Análisis léxico en compiladores	`[A-Za-z_][A-Za-z0-9_]*` para identificadores; `[0-9]+(\.[0-9]+)?` para números	Un generador como `lex` convierte cada patrón en un AFN, los une, determiniza y produce un AFD que lee el código una sola vez y devuelve la ficha más larga que coincide (Hopcroft et al., 2007, sec. 3.3.2).
Búsqueda de patrones en texto	`grep -E 'colou?r'`	`u?` equivale a $(u + \lambda)$: encuentra «color» y «colour» en todas las líneas de un archivo.
Validación de datos	`^[0-9]{5}$` para un código postal mexicano	Exige exactamente cinco dígitos y nada más; un formulario rechaza la entrada antes de enviarla.

Aplicación	Expresión concreta	Cómo funciona
Filtrado de registros	`^\S+ \S+ \S+ \[[^]]+\] "GET [^"]*" 404`	Sobre una bitácora de un servidor web en formato común, selecciona las peticiones GET que respondieron 404 (recurso no encontrado).
Automatización en el sistema operativo	`sed -E 's/([0-9]{4})-([0-9]{2})-([0-9]{2})/\3/\2/\1/'`	Reescribe fechas ISO 8601 (2026-11-06) al formato 06/11/2026 en todos los archivos de un lote; los grupos capturan año, mes y día.

En los cinco casos la parte que decide si un texto coincide es regular; la sustitución con `\3/\2/\1` del último usa los grupos para *escribir* la salida, no para reconocer, así que no sale de la clase regular. Los validadores del Ejercicio 4 aplican esta idea con autómatas propios.

3. Ejercicio 2. De expresión regular a autómata

3.1. Notación aceptada

El analizador (`src/nucleo/expresiones.py`) acepta la notación del curso:

Tabla 3. Notación del analizador. Los espacios se ignoran.

Escritura	Significado
`a`, `0`, `@`, …	Cualquier carácter que no sea operador es un símbolo.
`\x`	El carácter x literal aunque sea operador (por ejemplo `\+`, `\(`).
λ o `ε`	Palabra vacía.
$\varnothing$ o `Ø`	Lenguaje vacío.
`r + s`	Unión (precedencia más baja).
`rs`	Concatenación por yuxtaposición.
`r*`	Cerradura de Kleene (precedencia más alta).
`( r )`	Agrupación.
`[D] [m] [M] [L] [X]`	Clases declaradas: dígitos 0–9, minúsculas a–z, mayúsculas A–Z, letras y letras con dígitos. Se pueden registrar más con `Clases.registrar`.

Una clase es una abreviatura de la unión de sus símbolos: [D] = 0 + 1 + … + 9.

3.2. Análisis: *shunting-yard* con posiciones

El análisis tiene cuatro pasos, y cada ficha conserva la posición del carácter que la originó:

1. **Léxico.** `Lexico` convierte el texto en fichas: símbolo, clase, λ, $\varnothing$, `+`, `*`, `(` y `)`.
2. **Concatenación implícita.** Entre una ficha que puede terminar un operando (símbolo, clase, λ, $\varnothing$, `)` o `*`) y una que puede empezarlo (símbolo, clase, λ, $\varnothing$ o `(`) se inserta un operador explícito $\cdot$. En este mismo recorrido se detectan los errores de forma: un `+` sin operando, una `*` que no se aplica a nada o paréntesis vacíos.
3. **Posfija.** El algoritmo *shunting-yard* de Dijkstra pasa la secuencia a notación posfija con una pila de operadores, respetando `*` > $\cdot$ > `+` y los paréntesis; aquí se detectan los paréntesis desbalanceados.
4. **Árbol.** La posfija se reduce con una pila de nodos: `Simbolo`, `Clase`, `Lambda`, `Vacio`, `Union`, `Concatenacion` y `Estrella`.

Cada nodo sabe escribirse en la notación del curso con los paréntesis mínimos (`notacion()`), traducirse a la sintaxis de `re` (`python()`), decir si es anulable y qué símbolos usa.

Tabla 4. Expresiones mal formadas y su mensaje.

Entrada	Mensaje
`(a + b`	Falta el «)» que cierra este paréntesis (posición 1)
`a + * b`	La «*» no se aplica a nada (posición 5)
`ab)`	«)» sin un «(» que lo abra (posición 3)
`a()`	Paréntesis vacíos; use λ para la palabra vacía (posición 2)
`[Z]a`	La clase «[Z]» no está declarada (posición 1)
`a +`	Al «+» le falta el operando derecho (posición 3)

En la interfaz la revisión ocurre **mientras se escribe**: el campo muestra el mensaje y debajo aparece la expresión con un `^` bajo el carácter culpable.

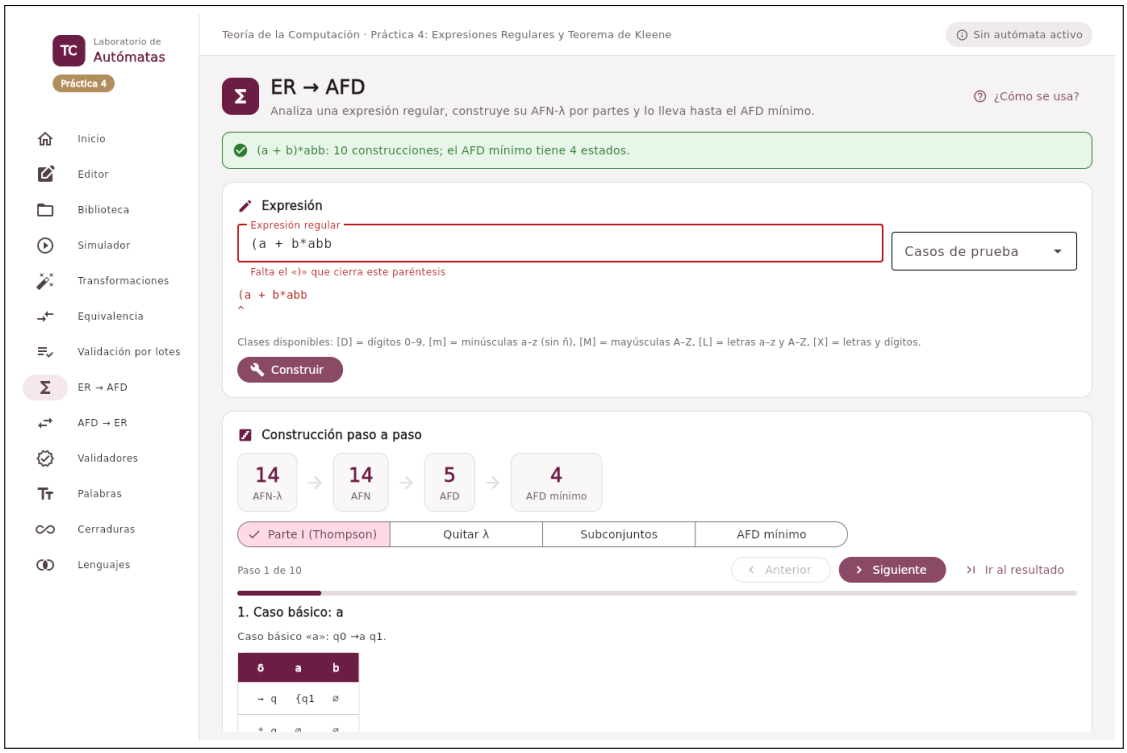


Figura 1. ER → AFD: error de sintaxis señalado con su posición.

3.3. Parte I del teorema de Kleene, paso a paso

`ConstruccionThompson` recorre el árbol en posorden y produce un **fragmento** (estado inicial, estado final, estados y aristas) por nodo. Los estados se numeran q0, q1, … en el orden en que se crean, de modo que un estado conserva su nombre en todas las etapas.

Tabla 5. Casos básicos y construcciones (Sipser, 2013, sec. 1.3, lema 1.55; Hopcroft et al., 2007, sec. 3.2.3).

Nodo	Construcción
$\varnothing$	Dos estados nuevos i, f sin transiciones.
λ	i $\to\lambda$ f.
a	i $\to$a f.
Clase	i $\to$ f con una transición por cada símbolo de la clase.
r + s	Nuevos i, f; i $\to\lambda$ inicio(r), i $\to\lambda$ inicio(s); fin(r) $\to\lambda$ f, fin(s) $\to\lambda$ f.
rs	fin(r) $\to\lambda$ inicio(s); el inicial es el de r y el final el de s.
r*	Nuevos i, f; i $\to\lambda$ inicio(r), i $\to\lambda$ f, fin(r) $\to\lambda$ inicio(r), fin(r) $\to\lambda$ f.

Cada construcción queda registrada como una etapa con su autómata parcial; la sección **ER**

→ **AFD** las recorre con «Anterior / Siguiente», dibuja cada fragmento y escribe qué pieza se está construyendo («Unión: a + b», «Estrella: (a + b)*»). Para (ab)*a (272b) las seis etapas son: caso básico a; caso básico b; concatenación ab; estrella (ab)*; caso básico a; concatenación (ab)*a, con 8 estados en total.

A ese AFN-λ se le aplican los algoritmos de la Práctica 3 —eliminación de λ, subconjuntos y minimización— y la misma barra de etapas permite ver cada uno. Cada autómata (AFN-λ, AFN, AFD y AFD mínimo) se exporta a `.jff` con *Guardar .jff* o los cuatro a la vez con *Guardar las cuatro*.

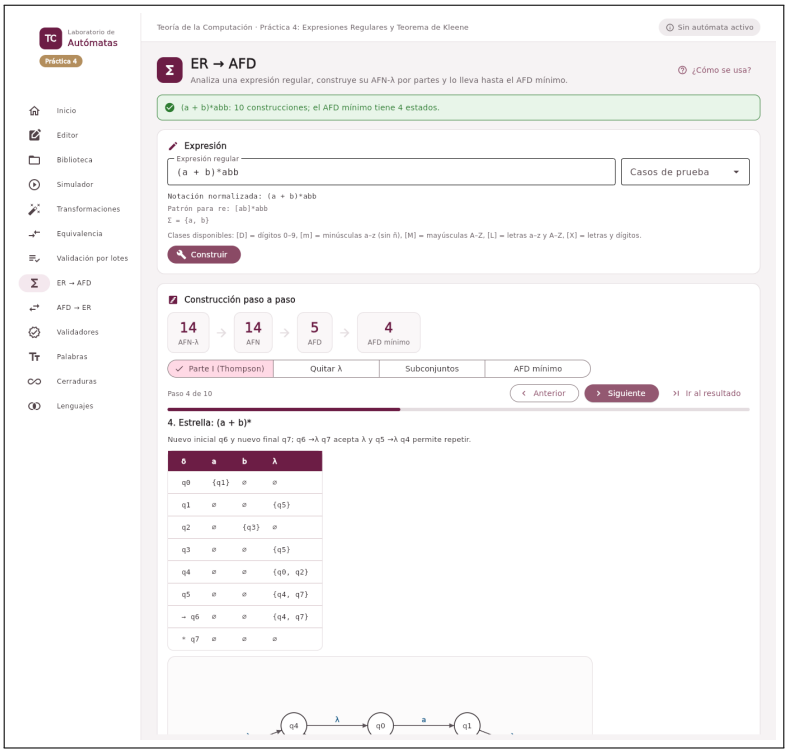


Figura 2. ER → AFD con (a + b)*abb: etapa «Estrella: (a + b)*» de la construcción.

3.4. Validación de palabras

La tarjeta *Probar palabras* valida mientras se escribe con el AFD mínimo y muestra también el resultado de `re.fullmatch` con la traducción de la expresión; si alguna vez difirieran, aparecería una advertencia.

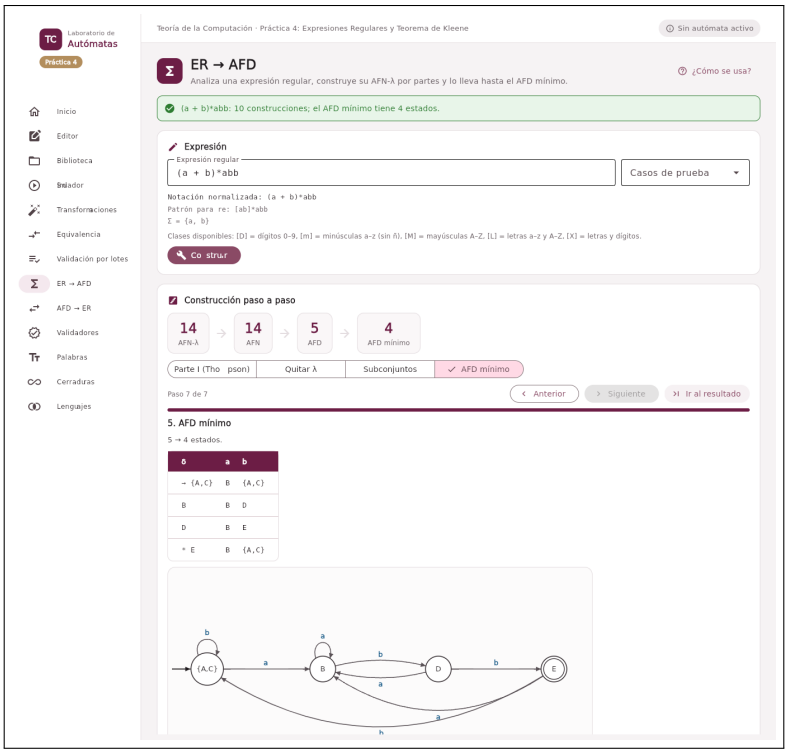


Figura 3. El AFD mínimo de (a + b)*abb y la prueba de la palabra `babb`.

3.5. Casos de prueba

Tabla 6. Tamaño de los autómatas en cada etapa (los AFD no incluyen el subconjunto ∅).

Caso	Expresión	Construcciones	AFN-λ	AFD	AFD mínimo
L3-241	0 + 1	3	6	3	2
L3-242	0* + 1	4	8	3	3
L3-243	(0 + 1)*	4	8	3	1
L3-244	0(0 + 1)	5	8	4	3
L3-245	0(0 + 1)*	6	10	4	2
L3-246	0*(0 + 1)	6	10	3	3
L3-247	(01)* + 1*	7	12	4	4
L3-248	(0* + 1)0	6	10	4	4
L3-249	(0* + 1)0*	7	12	4	2
L3-250	0*1* + 1*0	10	16	6	5
L3-251	(0 + 1*)0 + 1*1	11	18	4	4

Caso	Expresión	Construcciones	AFN-λ	AFD	AFD mínimo
L3-252	(0 + 1)*(00 + 11) + (00 + 11)(0 + 1)*	25	38	13	8
L3-253	(0 + (11 + 00)*)(11 + 10 + 01)*(λ + 110)	31	44	22	17
L4-272a	a + b*	4	8	3	3
L4-272b	(ab)*a	6	8	3	2
L4-272c	(a + b)*abb	10	14	5	4
L4-272d	a(ba)* + b	8	12	5	4
L4-272e	(a + λ)(b + $\varnothing$)	7	12	3	3
L4-278	(a + b)*b	6	10	3	2

Lista 4, 273. Procesando por λ-clausuras el AFN-λ de (ab)*a se aceptan `aba` y se rechazan `ab` y `abab`; con el de (a + b)*abb se acepta `babb` y se rechazan `aba`, `ab` y `abab`. Por ejemplo, en el segundo la lectura de `babb` empieza en λ-cl(q6) = {q0, q2, q4, q6, q7, q8} y termina en un conjunto que contiene al final q13.

Lista 4, 278. Para (a + b)*b: el AFN-λ tiene 10 estados; al quitar λ queda un AFN de 10 estados; la construcción de subconjuntos da 3 estados alcanzables y la minimización 2 (uno para «la última letra fue b», final, y otro para el resto). La eliminación de estados sobre ese AFD mínimo da a*b(b + aa*b)* y el lema de Arden (a + bb*a)*bb*; las dos son equivalentes a (a + b)*b según la prueba de equivalencia, con lo que se cierra el ciclo.

Pruebas. `tests/test_expresiones.py` (81 pruebas) comprueba la precedencia, la notación con paréntesis mínimos (y que es estable al volver a analizarla), quince expresiones mal formadas con su posición exacta, la traducción a `re`, la forma de los autómatas de Thompson (un final sin salidas, a lo sumo dos estados por nodo) y, para cada uno de los 19 casos, que **el AFN-λ, el AFN, el AFD y el AFD mínimo coinciden en todas las palabras de longitud $\leq$ 10** y que el AFD mínimo coincide con `re.fullmatch` en todas las de longitud ≤ 7.

4. Ejercicio 3. De autómata a expresión regular

4.1. Verificación del determinismo

La sección **AFD → ER** abre el autómata activo o cualquier `.jff` de la biblioteca y, antes de convertir, llama a `verificar_determinismo`, que devuelve qué pares (estado, símbolo) impiden que sea un AFD: los que tienen dos o más destinos y los estados con transiciones λ. Por ejemplo, para `no-determinista-263.jff` el mensaje es «No es determinista en 1 pares: δ(q0, 0) = {q0, q1}», y el botón de conversión indica que primero hay que pasar por *Transformaciones*.

4.2. Eliminación de estados paso a paso

`AutomataGeneralizado` es un autómata cuyas aristas llevan expresiones: hay una sola arista por par ordenado (las paralelas se unen con +), un inicial nuevo I con I $\to\lambda$ q_0 y un final nuevo T al que llega cada estado de F con λ. `EliminacionDeEstados` quita los estados originales uno a uno; al quitar k, para cada p → k → q:

$$R(p,q) \leftarrow R(p,q) + R(p,k)\,R(k,k)^{*}\,R(k,q).$$

Las expresiones se combinan con constructores que aplican sólo leyes seguras ($\varnothing$ absorbente, λ neutro, idempotencia, (r*)* = r*, (λ + r)* = r*), para que el resultado no crezca con términos inútiles. El orden puede fijarse escribiéndolo («q1, q2, q0») o dejarse al programa, que elige en cada paso el estado que crea menos aristas nuevas (entrantes × salientes). Cada eliminación es una etapa: la interfaz muestra la tabla «De · A · Expresión» del autómata generalizado que queda y la explicación con cada R(p, q) recalculado, y se avanza con «Siguiente». Como segundo método, `SistemaArden` plantea $X_q = \sum_a a\,X_{\delta(q,a)}$ (más λ si q ∈ F) y despeja con el lema de Arden, X = AX + B ⇒ X = A*B, mostrando el sistema después de cada sustitución.

4.3. Comparación con JFLAP

En la Práctica 3 se obtuvieron con JFLAP las expresiones de cinco AFD (119, 122, 138, 140b y 142a). Con el mismo orden de eliminación, el software produce exactamente las mismas expre-

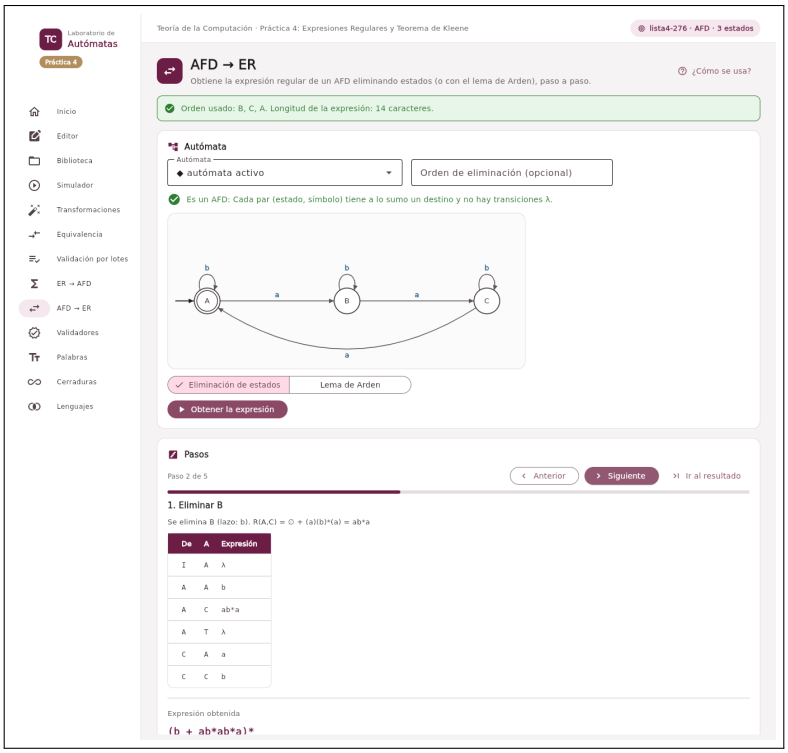


Figura 4. AFD → ER con el AFD 276: autómata generalizado después de eliminar B.

siones que JFLAP; sólo cambian los espacios. Para comprobar que la comparación sirve cuando la escritura no coincide, en JFLAP se repitió la conversión con otro orden: las expresiones cambian de forma, pero describen el mismo lenguaje. Para decidirlo, ambas expresiones se convierten con el Ejercicio 2 en AFD mínimos y se comparan con la prueba de equivalencia de la Práctica 3: la tarjeta *Comparación* de la sección hace exactamente eso al pegar la expresión de JFLAP.

Tabla 7. Comparación con los resultados de JFLAP.

AFD	JFLAP, orden de la Práctica 3	JFLAP, otro orden	Software (orden automático)	¿Equivalentes?
119	(b + a(b + ab))*(λ + a)	(b + ab + aab)*(λ + a) (q1, q2, q3)	(b + a(b + ab))*(λ + a)	Sí
122	(0 + (1 + 2)(0 + (1 + 2)0))*(λ + (1 + 2)(λ + 1 + 2))	(0 + (1 + 2)0 + (1 + 2)(1 + 2)0)*(λ + 1 + 2 + (1 + 2)(1 + 2)) (q1, q2, q3)	(0 + (1 + 2)(0 + (1 + 2)0))*(λ + (1 + 2)(λ + 1 + 2))	Sí
138	(a + ba*ba*b)*	(a + ba*ba*b)* (q2, q1)	(a + ba*ba*b)*	Sí

AFD	JFLAP, orden de la Práctica 3	JFLAP, otro orden	Software (orden automático)	¿Equivalentes?
140b	(00 + 11 + (01 + 10)(11 + 00)*(10 + 01))*	(1(00)*1 + (0 + 1(00)*01)(11 + 10(00)*01)*(0 + 10(00)*1))* (q3, q2, q1)	(00 + 11 + (01 + 10)(11 + 00)*(10 + 01))*	Sí
142a	(ab)*	(ab)* (q1, q2)	(ab)*	Sí

Con el mismo orden las expresiones son idénticas. Con otro orden cambian de forma: en 119 y 122, quitar q1 primero distribuye la concatenación, y en 140b, quitar q3 primero produce una expresión con (00)* que no se parece a la original. En 138 y 142a el resultado no depende del orden. En los cinco casos los AFD mínimos coinciden salvo el nombre de los estados.

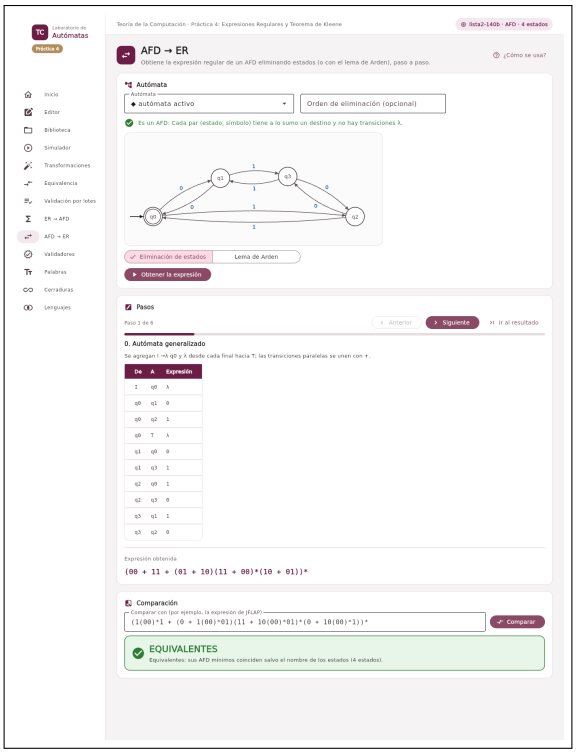


Figura 5. Comparación de la expresión del software con la que da JFLAP para el AFD 140b eliminando en el orden q3, q2, q1.

4.4. Casos de prueba

Tabla 8. Expresiones obtenidas por los dos métodos.

Caso	Orden elegido	Eliminación de estados	Lema de Arden
L3-254	A, B	0*1(1 + 00*1)*	(0 + 11*0)*11*
L3-255	B, A	((0 + 1)1*0)*	((0 + 1)1*0)*
L3-256	A, B	(1 + 2)*0(0 + 1 + 2(1 + 2)*0)*	(1 + 2 + 0(0 + 1)*2)*0(0 + 1)*
L3-257	B, C, A	((1 + 01*0)(0 + 1))*(01* + 1 + 01*0)	(1(0 + 1) + 01*0(0 + 1))*(1 + 01*(λ + 0))
L3-258	B, A, C	(0 + 1)(01 + (1 + 00)(0 + 1))*	((0 + 1)1 + (0 + 1)0(10)*(0 + 11))*(0 + 1 + (0 + 1)0(10)*1)
L4-276	B, C, A	(b + ab*ab*a)*	(b + ab*ab*a)*
L4-277	A, B	0*1(1 + 00*1)*	(0 + 11*0)*11*

Lista 4, 276 (número de aes múltiplo de 3). Arden: $X_A = bX_A + aX_B + \lambda$, $X_B = bX_B + aX_C$, $X_C = bX_C + aX_A$. Por Arden, $X_C = b^*aX_A$; sustituyendo, $X_B = bX_B + ab^*aX_A \Rightarrow X_B = b^*ab^*aX_A$; y $X_A = (b + ab^*ab^*a)X_A + \lambda \Rightarrow X_A = (b + ab^*ab^*a)^*$.

Lista 4, 277 (cadenas que terminan en 1). Las dos expresiones parecen distintas: 0*1(1 + 00*1)* y (0 + 11*0)*11*. Su equivalencia se justifica con leyes: 1 + 00*1 = (λ + 00*)1 = 0*1, así que la primera es 0*1(0*1)* = (0*1)⁺, las palabras formadas por bloques que terminan en 1; y (0 + 11*0)*11* = (0 + 1⁺0)*1⁺ es una sucesión de bloques que terminan en 0 seguida de al menos un 1. Ambas describen exactamente las palabras no vacías que terminan en 1, lo que confirma la prueba de equivalencia por autómatas mínimos. Como comprobación adicional, las dos aceptan `1`, `01`, `0011` y `1101`, y rechazan λ, `0`, `10` y `110`; esos ejemplos por sí solos no prueban la equivalencia.

Pruebas. `tests/test_kleene.py` (48 pruebas) convierte cada uno de los catorce autómatas (254–258, 276, 277 y los cinco de la Práctica 3) a expresión por eliminación —en orden automático y en el orden de Q— y por Arden, vuelve a construir el autómata de cada expresión y exige que sea equivalente al original. También comprueba el mensaje de no determinismo, la unión de aristas paralelas, que los nombres I y T no choquen con estados existentes, el lenguaje vacío y la comparación de pares de expresiones equivalentes y no equivalentes.

5. Ejercicio 4. Aplicaciones: validadores

Se implementaron tres validadores: **correo electrónico**, **teléfono en formato nacional** y **URL**. Cada uno es un objeto `Validador` (`src/nucleo/validadores.py`) que guarda la expresión en la notación del curso; la traducción a `re` se genera del árbol sintáctico, y el texto se valida con el **AFD mínimo** que produce el Ejercicio 2 (Thompson, eliminación de λ, subconjuntos y minimización). La sección **Validadores** muestra la expresión, su traducción, el número de estados antes y después de minimizar, el diagrama del AFD mínimo y la lista de lo que queda fuera; el texto se valida mientras se escribe e indica en qué carácter falla.

Los alfabetos grandes se escriben con las clases del Ejercicio 2: [D] (dígitos), [m] (minúsculas), [L] (letras) y [X] (letras y dígitos).

5.1. Correo electrónico

Notación del curso.

```
([X] + _)([X] + _ + -)*(.([X] + _)([X] + _ + -)*)*@[X]([X] + -)*(.[X]([X] +
↪  -)*)*.[L][L][L]*
```

Módulo `re`.

```
⌋
↪  [0-9A-Z_a-z][\-0-9A-Z_a-z]*(?:\.[0-9A-Z_a-z][\-0-9A-Z_a-z]*)*@[0-9A-Za-z][\-0-9A-Za-z]*(?:\.[0-9
```

La parte local son palabras (letras, dígitos, `_` y `-`, sin empezar con guion) separadas por un punto; después vienen `@`, uno o más subdominios y un dominio de primer nivel de al menos dos letras. Válidos: `juan.rico@ipn.mx`, `j_diego-2025@alumno.ipn.mx`, `a@b.co`; inválidos: `juan..rico@ipn.mx` (dos puntos seguidos), `@ipn.mx`, `juan@ipn` (sin dominio de primer nivel), `.juan@ipn.mx`.

Norma consultada y lo que queda fuera. Se tomó como referencia la forma *Dot-string* de la parte local del RFC 5321 (Klensin, 2008, sec. 4.1.2) y la sintaxis de etiquetas de dominio del RFC

1035 (Mockapetris, 1987, sec. 2.3.1). Quedan fuera, a propósito: las partes locales entre comillas y caracteres como `! # $% & ' * + / = ? ^ { | } ~` que el estándar sí admite; los dominios internacionalizados (con ñ o acentos) y las direcciones IP entre corchetes. Al revés, la expresión acepta etiquetas que terminan en guion o que pasan de 63 caracteres, que el RFC 1035 prohíbe. Una expresión regular para correos siempre es una aproximación: la gramática completa del estándar admite comentarios anidados, que no son regulares.

5.2. Teléfono en formato nacional (México)

Notación del curso.

```
(\+52 + λ)([D][D](-[D][D][D][D]-[D][D][D][D] + .[D][D][D][D].[D][D][D][D] +
↪  [D][D][D][D][D][D][D][D])
         + [D][D][D]-[D][D][D]-[D][D][D][D] + \([D][D]\)[D][D][D][D]-[D][D][D][D])
```

Módulo `re`.

```
⌋
↪  (?:\+52|)(?:(?:[0-9][0-9](?:(?:\-[0-9][0-9][0-9][0-9]\-[0-9][0-9][0-9][0-9]|\.[0-9][0-9][0-9][0-9
```

Diez dígitos, con el prefijo de país +52 opcional, en cinco presentaciones: sin separadores, 2-4-4 con guiones o con puntos, 3-3-4 con guiones y la clave de dos dígitos entre paréntesis. La expresión obliga a usar el **mismo** separador en todo el número. Válidos: `5512345678`, `55-1234-5678`, `+52222-123-4567`, `(55)1234-5678`, `55.1234.5678`; inválidos: `55-1234.5678` (separadores mezclados), `551234567` (nueve dígitos), `55 1234 5678`, `(55)12345678`.

Norma consultada y lo que queda fuera. El Plan Técnico Fundamental de Numeración del Instituto Federal de Telecomunicaciones, que desde 2019 fija la marcación nacional a 10 dígitos (Instituto Federal de Telecomunicaciones, 2019). Quedan fuera los números con espacios (el alfabeto del autómata no incluye el espacio, para no confundirlo con el separador de los formatos de archivo), los prefijos antiguos 01, 044 y 045, y cualquier verificación de que la clave de larga distancia exista: la expresión acepta cualquier combinación de dígitos.

5.3. URL (http y https)

Notación del curso.

```
http(s + λ)://([m] + [D])([m] + [D] + -)*(.([m] + [D])([m] + [D] +
↪  -)*)*.[m][m][m]*(:[D][D]* + λ)(/([X] + - + _ + . + ~)*)*
```

Módulo `re`.

```
⌋
↪  http(?:s|)://[0-9a-z][\-0-9a-z]*(?:\.[0-9a-z][\-0-9a-z]*)*\.[a-z][a-z][a-z]*(?::[0-9][0-9]*|)(?:/
```

Esquema `http` o `https`, nombre de host en minúsculas con dominio de primer nivel, puerto opcional y una ruta de segmentos con caracteres no reservados. Válidos: `https://www.ipn.mx`, `http://escom.ipn.mx:8080/tc/practica4`, `https://a.io/`; inválidos: `ftp://ipn.mx`, `https//ipn.mx`, `https://ipn`, `https://IPN.mx`, `https://ipn.mx:/x`.

Norma consultada y lo que queda fuera. RFC 3986, secciones 3.1 a 3.3 (*scheme*, *authority* y *path*) (Berners-Lee et al., 2005). Quedan fuera las consultas (`?a=1`), los fragmentos (`#sec`), los caracteres codificados con `%`, los hosts en mayúsculas o como direcciones IP, la información de usuario (`usuario@host`) y los demás esquemas.

5.4. Tamaño de los autómatas

Tabla 9. Estados de cada validador antes y después de la minimización.

Validador	$\|\Sigma\|$	AFN-λ (Thompson)	AFD (subconjuntos)	AFD mínimo
Correo	66	76	23	7
Teléfono	15	128	56	27
URL	68	106	31	16

La minimización reduce el correo de 23 a 7 estados porque los subconjuntos que sólo difieren en qué copia de la clase [X] están activas se comportan igual; el teléfono conserva más estados porque cada posición de los diez dígitos, en cada formato, es una situación distinta.

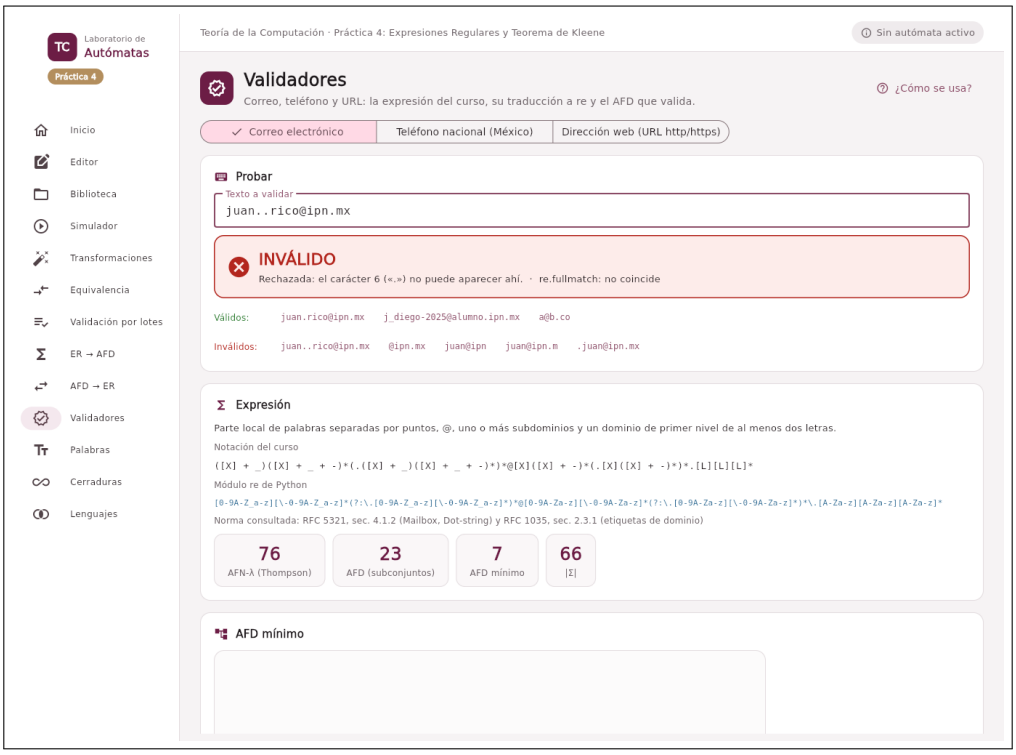


Figura 6. Validador de correo: texto inválido con la posición del error, expresión, traducción y conteo de estados.

5.5. Congruencia entre modelos

`tests/test_validadores.py` compara, para cada validador, el AFD con `re.fullmatch` (Python Software Foundation, 2026) sobre un conjunto generado por `GeneradorDePruebas`:

1. **300 palabras válidas** obtenidas recorriendo el árbol de la expresión al azar (con semilla fija, para que la prueba sea reproducible): en una unión se elige una rama, en una clase un símbolo y en una estrella entre 0 y 4 repeticiones.
2. **300 mutaciones** de esas palabras: una o dos operaciones elegidas al azar entre insertar, borrar, sustituir o duplicar un carácter, o intercambiar dos vecinos. La mitad de los caracteres nuevos salen del alfabeto del autómata y la otra mitad de un «ruido» fijo con espacio, ñ y signos (`@ .`

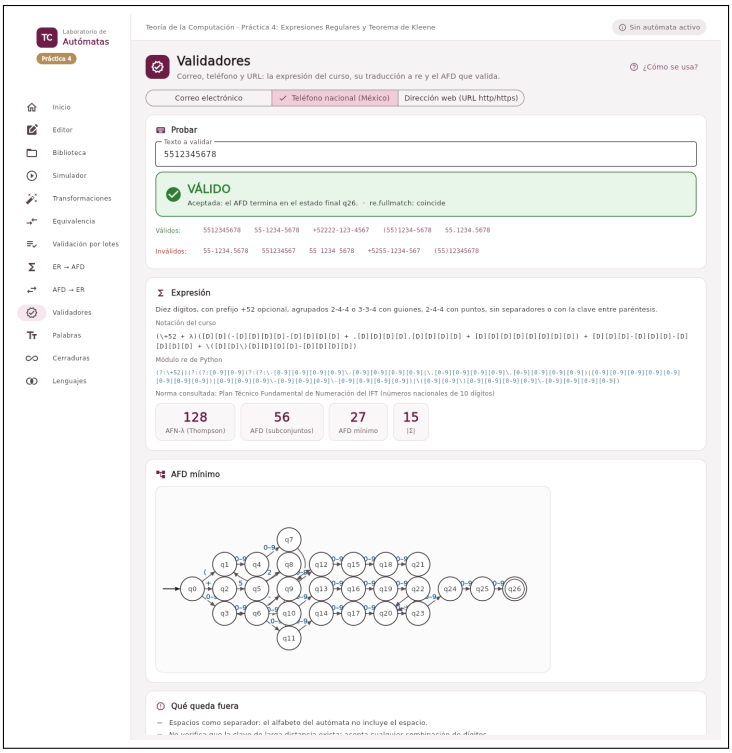


Figura 7. Validador de teléfono y su AFD mínimo.

`-` `_` `/` `:` `+` `(` `)` `#`).

3. Los **casos frontera** escritos a mano: la cadena vacía, un espacio y los ejemplos válidos e inválidos de cada validador.

Tras quitar repetidas quedan más de 600 palabras por validador, con al menos 100 válidas y 100 inválidas según `re` (en la última ejecución: correo 464 / 143, teléfono 328 / 281, URL 432 / 175). El autómata y `re.fullmatch` coinciden en **todas**. Las pruebas pasan con Python 3.11, 3.12 y 3.13.

6. Conclusiones

La investigación dejó clara la frontera entre la expresión regular de la teoría y la de los lenguajes de programación: las clases y los cuantificadores sólo abrevian, pero una referencia hacia atrás como (a*)b\1 describe $\{a^n b a^n\}$, que el lema de bombeo prueba no regular. Esa frontera justifica validar con un autómata y no con un motor de retroceso.

Con las dos direcciones del teorema de Kleene implementadas, el software cierra el ciclo: una

expresión se convierte en AFN-λ por partes, se lleva hasta el AFD mínimo con los algoritmos de la Práctica 3 y de ahí regresa a una expresión por eliminación de estados o por el lema de Arden. Analizar con *shunting-yard* y conservar la posición de cada ficha permitió señalar los errores en el carácter exacto mientras se escribe, y registrar cada construcción como una etapa hizo que la interfaz mostrara la Parte I del teorema pieza por pieza sin código especial.

Comparar las expresiones del software con las de JFLAP mostró que dos expresiones distintas en su escritura pueden describir el mismo lenguaje, y que la forma correcta de decidirlo es convertirlas en AFD mínimos; leer la igualdad a simple vista, como en 0*1(1 + 00*1)* y (0 + 11*0)*11*, es poco confiable. Los tres validadores, comparados con `re.fullmatch` sobre más de 600 palabras cada uno, coincidieron en todas, y la minimización redujo sus autómatas hasta en 70 %; el ejercicio también mostró que toda expresión para correos o URL es una aproximación consciente al estándar, con exclusiones que hay que documentar.

Referencias

Berners-Lee, T., Fielding, R. T., & Masinter, L. (2005). *Uniform Resource Identifier (URI): Generic syntax* (RFC N.º 3986). Internet Engineering Task Force. https://doi.org/10.17487/RFC3986

Câmpeanu, C., Salomaa, K., & Yu, S. (2003). A formal study of practical regular expressions. *International Journal of Foundations of Computer Science*, *14*(6), 1007-1018.

Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory*, *2*(3), 113-124. https://doi.org/10.1109/TIT.1956.1056813

Davis, J. C., Coghlan, C. A., Servant, F., & Lee, D. (2018). The impact of regular expression denial of service (ReDoS) in practice: An empirical study at the ecosystem scale. *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 246-256. https://doi.org/10.1145/3236024.3236027

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introduction to automata theory, languages, and computation* (3.ª ed.). Pearson Addison-Wesley.

Instituto Federal de Telecomunicaciones. (2019). *Acuerdo mediante el cual el Pleno del Instituto Federal de Telecomunicaciones modifica el Plan Técnico Fundamental de Numeración, publicado en el Diario Oficial de la Federación el 11 de mayo de 2018*. Consultado el 6 de noviembre de 2026, desde https://www.ift.org.mx/sites/default/files/industria/temasrelevantes/14298/documentos/acuerdomodifptfnconvivenciaengroseucs100719limpia.pdf

Kleene, S. C. (1956). Representation of events in nerve nets and finite automata. En C. E. Shannon & J. McCarthy (Eds.), *Automata Studies* (pp. 3-41). Princeton University Press. https://doi.org/10.1515/9781400882618-002

Klensin, J. (2008). *Simple Mail Transfer Protocol* (RFC N.º 5321). Internet Engineering Task Force. https://doi.org/10.17487/RFC5321

Mockapetris, P. (1987). *Domain names: Implementation and specification* (RFC N.º 1035). Internet Engineering Task Force. https://doi.org/10.17487/RFC1035

Python Software Foundation. (2026). *re: Regular expression operations (Python 3.12 documentation)*. Consultado el 6 de noviembre de 2026, desde https://docs.python.org/3.12/library/re.html

Sipser, M. (2013). *Introduction to the theory of computation* (3.ª ed.). Cengage Learning.